

1. Cálculo de Salário com Reajuste

Entrada: salário (sal), reajuste (readj)

Processamento:

- Ler salário e reajuste em loop até que um valor negativo seja informado
- Calcular o salário reajustado chamando a função `add`
- Classificar o resultado: abaixo da média (< 1400), dentro da média (≤ 1800), acima da média (> 1800)

Saída: Salário reajustado e sua classificação

Algoritmo

1. Declarar e inicializar as variáveis `sal = 0.0`, `readj = 0.0`, `result = 0.0` do tipo real.
2. Apresentar mensagem de instrução: "Para parar o programa, digite qualquer valor negativo."
3. Apresentar linha separadora.
4. Enquanto `True`, executar:
 1. Tentar ler um valor real e armazenar em `sal`.
 2. Se `sal` for menor que 0, interromper o loop.
 3. Tentar ler um valor real e armazenar em `readj`.
 4. Se `readj` for menor que 0, interromper o loop.
 5. Em caso de erro na leitura (valor inválido), apresentar mensagem de entrada inválida.
 6. Executar a função `add` passando `sal` e `readj` como parâmetros e armazenar o retorno em `result`.
 7. Se `result` for menor que 1400, apresentar "Salário abaixo da média."
 8. Senão, se `result` for menor ou igual a 1800, apresentar "Salário dentro da média."
 9. Senão, apresentar "Salário acima da média."

Função `add`:

Receber como parâmetros `var1` e `var2` do tipo real.

Declarar e inicializar a variável `sum` do tipo real.

Calcular a soma de `var1` e `var2` e armazenar em `sum`.

Retornar o valor calculado e armazenado em `sum`.

2. Conceito de Notas

Entrada: 3 notas inteiras

Processamento:

- Ler 3 notas do usuário
- Calcular a média aritmética ponderada: $(\text{nota1} + \text{nota2} \times 2 + \text{nota3} \times 3) / 6$
- Chamar a função `find_concept` para obter o conceito (1 = A, 2 = B, 3 = C, 4 = D, 5 = E)
- Apresentar o conceito por extenso

Saída: Conceito A, B, C, D ou E

Algoritmo

1. Declarar e inicializar as variáveis `grades = []` do tipo lista, `ma = 0.0` do tipo real, `i = 0` do tipo inteiro.
2. Enquanto `i` for menor que 3, executar:
 1. Tentar ler um valor inteiro e armazenar na posição `i` da lista `grades`.
 2. Incrementar `i` em 1.
 3. Em caso de erro na leitura (valor inválido), apresentar mensagem de entrada inválida.
3. Calcular `ma` como $(\text{grades}[0] + \text{grades}[1] * 2 + \text{grades}[2] * 3) / 6$.
4. Executar a função `find_concept` passando `ma` como parâmetro e armazenar o retorno em `concept`.
5. Se `concept` for igual a 1, apresentar "Conceito A".
6. Senão, se `concept` for igual a 2, apresentar "Conceito B".
7. Senão, se `concept` for igual a 3, apresentar "Conceito C".
8. Senão, se `concept` for igual a 4, apresentar "Conceito D".
9. Senão, apresentar "Conceito E".

Função `find_concept`:

Receber como parâmetro `var` do tipo real.

Declarar e inicializar a variável `result` do tipo inteiro.

Se `var` for maior ou igual a 9, atribuir 1 a `result`.

Senão, se `var` for maior ou igual a 7.5, atribuir 2 a `result`.

Senão, se `var` for maior ou igual a 6, atribuir 3 a `result`.

Senão, se `var` for maior ou igual a 4, atribuir 4 a `result`.

Senão, atribuir 5 a `result`.

Retornar o valor armazenado em `result`.

3. Maior Número

Entrada: 3 números inteiros

Processamento:

- Ler 3 números do usuário
- Chamar a função `find_max` para encontrar o maior valor
- Apresentar o resultado

Saída: O maior número informado

Algoritmo

1. Declarar e inicializar a variável `nums = []` do tipo lista.
2. Para `i` no intervalo de 0 até 2, executar:
 1. Ler um valor inteiro e armazenar na posição `i` da lista `nums`.
3. Executar a função `find_max` passando `nums` como parâmetro e armazenar o retorno em `max`.
4. Apresentar "O maior número é: " e o valor de `max`.

Função `find_max`:

Receber como parâmetro `var` do tipo lista.

Declarar e inicializar a variável `biggest` do tipo inteiro com o valor do primeiro elemento de `var`.

Para cada elemento `i` em `var` a partir do segundo elemento, executar:

Se `i` for maior que `biggest`, atribuir `i` a `biggest`.

Retornar o valor armazenado em `biggest`.

4. Desconto/Acréscimo por Parcelas

Entrada: valor de venda (`sell_val`), quantidade de parcelas (`installment_qty`)

Processamento:

- Ler valor de venda e quantidade de parcelas
- Chamar a função `get_installment_rate` para obter a taxa conforme a quantidade de parcelas

- Chamar a função `calc_percentage` para calcular o valor do desconto/acrécimo
- Calcular o valor final somando o valor original com o desconto/acrécimo

Saída: Valor do desconto/acrécimo e valor final

Algoritmo

1. Declarar e inicializar as variáveis `sell_val = 0.0`, `installment = 0.0`, `discount = 0.0` do tipo real e `installment_qty = 0` do tipo inteiro.
2. Ler um valor real e armazenar em `sell_val`.
3. Ler um valor inteiro e armazenar em `installment_qty`.
4. Executar a função `get_installment_rate` passando `installment_qty` como parâmetro e armazenar o retorno em `installment`.
5. Executar a função `calc_percentage` passando `sell_val` e `installment` como parâmetros e armazenar o retorno em `discount`.
6. Apresentar "Valor do desconto/acrécimo: " e o valor de `discount`.
7. Apresentar "Valor final com desconto/acrécimo: " e o resultado de `sell_val + discount`.

Função `get_installment_rate`:

Receber como parâmetro `var` do tipo inteiro.
 Declarar e inicializar a variável `installment` do tipo real.
 Se `var` for igual a 1, atribuir -5.0 a `installment`.
 Senão, se `var` for igual a 2, atribuir 1.0 a `installment`.
 Senão, se `var` for igual a 3, atribuir 4.5 a `installment`.
 Senão, se `var` for igual a 4, atribuir 7.5 a `installment`.
 Senão, atribuir 10.0 a `installment`.
 Retornar o valor armazenado em `installment`.

Função `calc_percentage`:

Receber como parâmetros `amount` do tipo real e `percentage` do tipo real.
 Declarar e inicializar a variável `value` do tipo real.
 Calcular $(amount * percentage) / 100$ e armazenar em `value`.
 Retornar o valor calculado e armazenado em `value`.

5. Média de Notas (Aritmética ou Ponderada)

Entrada: 3 notas (`grade1`, `grade2`, `grade3`), tipo de média (`grade_type` — A ou P)

Processamento:

- Ler 3 notas do usuário
- Ler o tipo de média (A = aritmética, P = ponderada)
- Chamar a função `calc_grade` para calcular a média conforme o tipo
- Apresentar o resultado

Saída: Média do aluno

Algoritmo

1. Declarar e inicializar as variáveis `grades = []` do tipo lista, `grade_type = ""` do tipo caractere, `i = 0` do tipo inteiro, `final_grade = 0.0` do tipo real.
2. Enquanto `i` for menor que 3, executar:
 1. Tentar ler um valor real e armazenar na posição `i` da lista `grades`.
 2. Incrementar `i` em 1.
 3. Em caso de erro na leitura (valor inválido), apresentar mensagem de valor inválido.
3. Enquanto `True`, executar:
 1. Tentar ler um valor caractere, converter para maiúsculo e armazenar em `grade_type`.
 2. Em caso de erro na leitura (valor inválido), apresentar mensagem de valor inválido.
 3. Se `grade_type` for igual a "A" ou "P", interromper o loop.
 4. Senão, apresentar mensagem de valor inválido.
4. Executar a função `calc_grade` passando `grades[0]`, `grades[1]`, `grades[2]` e `grade_type` como parâmetros e armazenar o retorno em `final_grade`.
5. Apresentar "A média do aluno é " e o valor de `final_grade`.

Função `calc_grade`:

Receber como parâmetros `grade1`, `grade2`, `grade3` do tipo real e `grade_type` do tipo caractere.

Declarar e inicializar a variável `grade` do tipo real.

Se `grade_type` for igual a "A", calcular $(grade1 + grade2 + grade3) / 3$ e armazenar em `grade`.

Senão, se `grade_type` for igual a "P", calcular $(grade1 * 5 + grade2 * 3 + grade3 * 2) / 10$ e armazenar em `grade`.

Retornar o valor calculado e armazenado em `grade`.

6. Categoria de Idade

Entrada: idade (age)

Processamento:

- Ler a idade do usuário
- Chamar a função `age_category` para classificar a idade em uma categoria
- Apresentar a categoria

Saída: Categoria da idade (Infantil A, Infantil B, Juvenil A, Juvenil B ou Adulto)

Algoritmo

1. Declarar e inicializar a variável `age = 0` do tipo inteiro.
2. Ler um valor inteiro e armazenar em `age`.
3. Executar a função `age_category` passando `age` como parâmetro e armazenar o retorno em `category`.
4. Apresentar o valor de `category`.

Função `age_category`:

Receber como parâmetro `age` do tipo inteiro.

Se `age` for maior ou igual a 5 e menor ou igual a 7, retornar "Infantil A".

Senão, se `age` for menor que 10, retornar "Infantil B".

Senão, se `age` for menor que 13, retornar "Juvenil A".

Senão, se `age` for menor que 17, retornar "Juvenil B".

Senão, retornar "Adulto".

7. Positivo ou Negativo

Entrada: um valor inteiro (val)

Processamento:

- Ler um valor do usuário
- Chamar a função `positive_or_negative` para verificar se o valor é positivo ou negativo
- Apresentar o resultado

Saída: Verdadeiro (positivo ou zero) ou Falso (negativo)

Algoritmo

1. Declarar e inicializar a variável `val = 0` do tipo inteiro.
2. Ler um valor inteiro e armazenar em `val`.
3. Executar a função `positive_or_negative` passando `val` como parâmetro e armazenar o retorno em `result`.
4. Apresentar o valor de `result`.

Função `positive_or_negative`:

Receber como parâmetro `var` do tipo inteiro.

Se `var` for menor que 0, retornar `False`.

Senão, retornar `True`.

8. Par ou Ímpar

Entrada: um valor inteiro (`val`)

Processamento:

- Ler um valor do usuário
- Chamar a função `even_or_uneven` para verificar se o valor é par ou ímpar
- Apresentar o resultado

Saída: Verdadeiro (par) ou Falso (ímpar)

Algoritmo

1. Declarar e inicializar a variável `val = 0` do tipo inteiro.
2. Ler um valor inteiro e armazenar em `val`.
3. Executar a função `even_or_uneven` passando `val` como parâmetro e armazenar o retorno em `result`.
4. Apresentar o valor de `result`.

Função `even_or_uneven`:

Receber como parâmetro `var` do tipo inteiro.

Se o resto da divisão de `var` por 2 for igual a 0, retornar `True`.

Senão, retornar `False`.

9. Fatorial

Entrada: um valor inteiro (`val`)

Processamento:

- Ler um valor do usuário
- Chamar a função `fatorial` para calcular o fatorial do valor
- Apresentar o resultado

Saída: Fatorial do valor informado

Algoritmo

1. Declarar e inicializar a variável `val = 0` do tipo inteiro.
2. Ler um valor inteiro e armazenar em `val`.
3. Executar a função `fatorial` passando `val` como parâmetro e armazenar o retorno em `result`.
4. Apresentar o valor de `result`.

Função `fatorial`:

Receber como parâmetro `var` do tipo inteiro.

Declarar e inicializar a variável `result = 1` do tipo inteiro.

Para `i` no intervalo de 1 até `var` (inclusive), executar:

Multiplicar `result` por `i` e armazenar em `result`.

Retornar o valor calculado e armazenado em `result`.

10. Reajuste Salarial

Entrada: salário (`salary`), quantidade de filhos (`children`)

Processamento:

- Ler o salário e a quantidade de filhos do funcionário
- Chamar a função `calc_readj` para calcular o salário reajustado
- Apresentar o resultado

Saída: Salário reajustado

Algoritmo

1. Declarar e inicializar as variáveis `salary = 0.0` do tipo real, `children = 0` do tipo inteiro.
2. Ler um valor real e armazenar em `salary`.
3. Ler um valor inteiro e armazenar em `children`.
4. Executar a função `calc_readj` passando `salary` e `children` como parâmetros e armazenar o retorno em `result`.
5. Apresentar "Salário reajustado: " e o valor de `result`.

Função `calc_readj`:

Receber como parâmetros `sal` do tipo real e `children` do tipo inteiro.

Declarar e inicializar a variável `readj` do tipo real.

Se `sal` for menor que 1000, atribuir 9.0 a `readj`.

Senão, se `sal` for menor que 3000, atribuir 7.0 a `readj`.

Senão, atribuir 1.0 a `readj`.

Se `children` for menor que 3, incrementar `readj` em 1.0.

Senão, incrementar `readj` em 2.0.

Calcular $(sal * readj) / 100$ e armazenar em `readj_sal`.

Somar `readj_sal` a `sal`.

Retornar o valor de `sal`.